Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Трекинг множественных объектов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Хомиченко И.А.

Проверила:

Андренко К.В.

Брест 2026

Цель: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов

Общее задание

1. Используя сеть-детектор, обученный в ЛР 3, реализовать логику для отслеживания множественных объектов, используя библиотеку Ultralytics YOLO;

2. Применять алгоритмы BoT-Sort и ByteTrack (задействовать соответствующие конфигурационные файлы);

3. Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;

4. В качестве исходных видеоматериалов для экспериментов использовать видео-ролики из сети (например, из YouTube), содержащие множественные объекты классов из ЛР 3;

5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Источники

1. Отслеживание нескольких объектов с помощью Ultralytics YOLO: https://docs.ultralytics.com/ru/modes/track/#features-at-a-glance

Код программы:

```
from __future__ import annotations

import argparse
import csv
import json
import time
from collections import defaultdict
from pathlib import Path

import cv2
import torch
import yaml
from ultralytics import YOLO

BASE = Path(__file__).resolve().parent


def options():
    parser = argparse.ArgumentParser()
    parser.add_argument('--source', default='videos')
```

```
    parser.add_argument('--weights', default=None)
    parser.add_argument('--device', default='auto')
    parser.add_argument('--imgsz', type=int, default=640)
    parser.add_argument('--conf', type=float, default=0.1)
    parser.add_argument('--limit', type=int, default=0)
    return parser.parse_args()


def find_weights(value):
    if value:
        path = Path(value).expanduser()
        path = path if path.is_absolute() else BASE / path
    else:
        latest = BASE / 'reports' / 'latest_run.json'
        if latest.exists():
            path = Path(json.loads(latest.read_text(encoding='utf-
8'))['best_weights'])
        else:
            candidates = sorted((BASE /
'runs').glob('train*/weights/best.pt'), key=lambda p: p.stat().st_mtime)
            if not candidates:
                raise FileNotFoundError('Не найден best.pt. Передайте --
weights путь/к/best.pt')
            path = candidates[-1]
    if not path.is_file():
        raise FileNotFoundError(f'Веса модели не найдены: {path}')
    return path.resolve()


def make_configs():
    common = {
        'track_high_thresh': 0.25,
        'track_low_thresh': 0.1,
        'new_track_thresh': 0.25,
        'track_buffer': 30,
        'match_thresh': 0.8,
        'fuse_score': True,
    }
    botsort = {
        **common,
        'tracker_type': 'botsort',
        'gmc_method': 'sparseOptFlow',
        'proximity_thresh': 0.5,
        'appearance_thresh': 0.8,
        'with_reid': False,
        'model': 'auto',
    }
    bytetrack = {**common, 'tracker_type': 'bytetrack'}
    variants = {
        'botsort_default': botsort,
        'botsort_tuned': {**botsort, 'track_buffer': 60, 'track_high_thresh':
0.4, 'new_track_thresh': 0.4},
        'bytetrack_default': bytetrack,
        'bytetrack_tuned': {**bytetrack, 'track_buffer': 60,
'track_high_thresh': 0.4, 'new_track_thresh': 0.4},
    }
    folder = BASE / 'configs'
    folder.mkdir(exist_ok=True)
    for name, data in variants.items():
        (folder / f'{name}.yaml').write_text(yaml.safe_dump(data,
sort_keys=False), encoding='utf-8')
    return variants
```

```
def analyze(video, name, config, weights, device, args):
    capture = cv2.VideoCapture(str(video))
    if not capture.isOpened():
        raise RuntimeError(f'Не удалось открыть видео: {video}')
    fps = capture.get(cv2.CAP_PROP_FPS)
    if not fps or fps <= 0:
        fps = 25.0
    width = int(capture.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(capture.get(cv2.CAP_PROP_FRAME_HEIGHT))
    if width <= 0 or height <= 0:
        capture.release()
        raise RuntimeError(f'Некорректный размер видео: {video}')
    folder = BASE / 'results' / video.stem / name
    folder.mkdir(parents=True, exist_ok=True)
    output = folder / 'tracked.mp4'
    writer = cv2.VideoWriter(str(output), cv2.VideoWriter_fourcc(*'mp4v'),
fps, (width, height))
    if not writer.isOpened():
        capture.release()
        raise RuntimeError(f'Не удалось создать видео: {output}')
    model = YOLO(str(weights))
    sightings = defaultdict(list)
    frame_index = 0
    detections = 0
    tracked_boxes = 0
    start = time.perf_counter()
    try:
        with (folder / 'tracks.csv').open('w', newline='', encoding='utf-8-
sig') as file:
            log = csv.writer(file)
            log.writerow(['frame', 'track_id', 'class', 'confidence', 'x1',
'y1', 'x2', 'y2'])
            while True:
                ok, frame = capture.read()
                if not ok or (args.limit and frame_index >= args.limit):
                    break
                frame_index += 1
                result = model.track(
                    frame, persist=True, tracker=str(config), device=device,
                    imgsz=args.imgsz, conf=args.conf, classes=[0],
verbose=False
                )[0]
                boxes = result.boxes
                if boxes is not None:
                    detections += len(boxes)
                    if boxes.is_track and boxes.id is not None:
                        ids = boxes.id.int().cpu().tolist()
                        coordinates = boxes.xyxy.cpu().tolist()
                        confidence = boxes.conf.cpu().tolist()
                        classes = boxes.cls.int().cpu().tolist()
                        for track_id, xyxy, score, cls in zip(ids,
coordinates, confidence, classes):
                            sightings[track_id].append(frame_index)
                            log.writerow([frame_index, track_id, cls,
round(score, 5), *[round(v, 2) for v in xyxy]])
                        tracked_boxes += len(ids)
                writer.write(result.plot())
    finally:
        capture.release()
        writer.release()
    elapsed = time.perf_counter() - start
    lengths = [len(frames) for frames in sightings.values()]
    return {
        'video': video.name,
```

```
        'tracker': name,
        'frames': frame_index,
        'seconds': round(elapsed, 3),
        'processing_fps': round(frame_index / elapsed, 2) if elapsed else 0,
        'detections': detections,
        'tracked_boxes': tracked_boxes,
        'unique_ids': len(sightings),
        'mean_track_frames': round(sum(lengths) / len(lengths), 2) if lengths
else 0,
        'longest_track_frames': max(lengths, default=0),
        'output': str(output.relative_to(BASE)),
    }


def main():
    args = options()
    if args.imgsz < 32 or args.imgsz % 32 or not 0 < args.conf < 1 or
args.limit < 0:
        raise ValueError('Неверные параметры imgsz, conf или limit')
    weights = find_weights(args.weights)
    device = 0 if args.device == 'auto' and torch.cuda.is_available() else
('cpu' if args.device == 'auto' else args.device)
    if str(device) != 'cpu' and not torch.cuda.is_available():
        raise RuntimeError('CUDA недоступна. Установите PyTorch с CUDA или
укажите --device cpu')
    print(f'Модель: {weights}\nУстройство: {device}\nCUDA:
{torch.cuda.is_available()}')
    variants = make_configs()
    source = Path(args.source).expanduser()

    source = source if source.is_absolute() else BASE / source
    extensions = {'.mp4', '.avi', '.mov', '.mkv', '.webm'}
    videos = [source] if source.is_file() else sorted(p for p in
source.iterdir() if p.is_file() and p.suffix.lower() in extensions) if
source.is_dir() else []
    if not videos:
        raise FileNotFoundError(f'Положите видео в папку {source} или укажите
--source путь/к/видео.mp4')
    rows = []
    for video in videos:
        for name in variants:
            print(f'Обработка: {video.name} / {name}')
            row = analyze(video, name, BASE / 'configs' / f'{name}.yaml',
weights, device, args)
            rows.append(row)
            print(f"  FPS={row['processing_fps']}; ID={row['unique_ids']};
средняя длина={row['mean_track_frames']} кадров")
    folder = BASE / 'results'
    with (folder / 'comparison.csv').open('w', newline='', encoding='utf-8-
sig') as file:
        writer = csv.DictWriter(file, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
    (folder / 'comparison.json').write_text(json.dumps(rows,
ensure_ascii=False, indent=2), encoding='utf-8')
    print(f'Готово. Результаты: {folder}')


if __name__ == '__main__':
    main()
```

Результат программы:

FPS=20.17; ID=112; средняя длина=110.46 кадров
Обработка: 6387-191695740_medium.mp4 / botsort_tuned
  FPS=28.56; ID=77; средняя длина=149.4 кадров
Обработка: 6387-191695740_medium.mp4 / bytetrack_default
  FPS=36.95; ID=111; средняя длина=110.56 кадров
Обработка: 6387-191695740_medium.mp4 / bytetrack_tuned
  FPS=36.75; ID=79; средняя длина=145.78 кадров



В ходе выполнения программы был реализован полный цикл: подготовка датасета из ZIP-экспорта Roboflow (конвертация полигонов в рамки, отсев ошибочной разметки), обучение модели YOLO12n на GPU NVIDIA RTX 3050 и оценка на тестовой выборке с расчётом precision, recall и mAP. Затем видео 6387-191695740_medium.mp4 обработано четырьмя конфигурациями трекеров (BoT-SORT и ByteTrack в дефолтном и настроенном вариантах) с замером FPS, числа уникальных ID и длины треков. Сравнение показало, что настройка параметров заметно повышает стабильность трекинга: число уникальных ID снизилось примерно на треть (BoT-SORT 112→77, ByteTrack 111→79), а средняя длина трека выросла со ~110 до ~147 кадров. При этом bytetrack_tuned работает быстрее всех (36.75 FPS) при качестве, почти не уступающем botsort_tuned (28.56 FPS, 77 ID, 149.4 кадра), поэтому для реального времени предпочтителен ByteTrack, для максимальной точности офлайн — BoT-SORT, а дефолтные конфиги проигрывают настроенным по всем показателям.

Вывод: **:** исследовал применение алгоритмов трекинга на базе обученной

сети-детектора объектов.